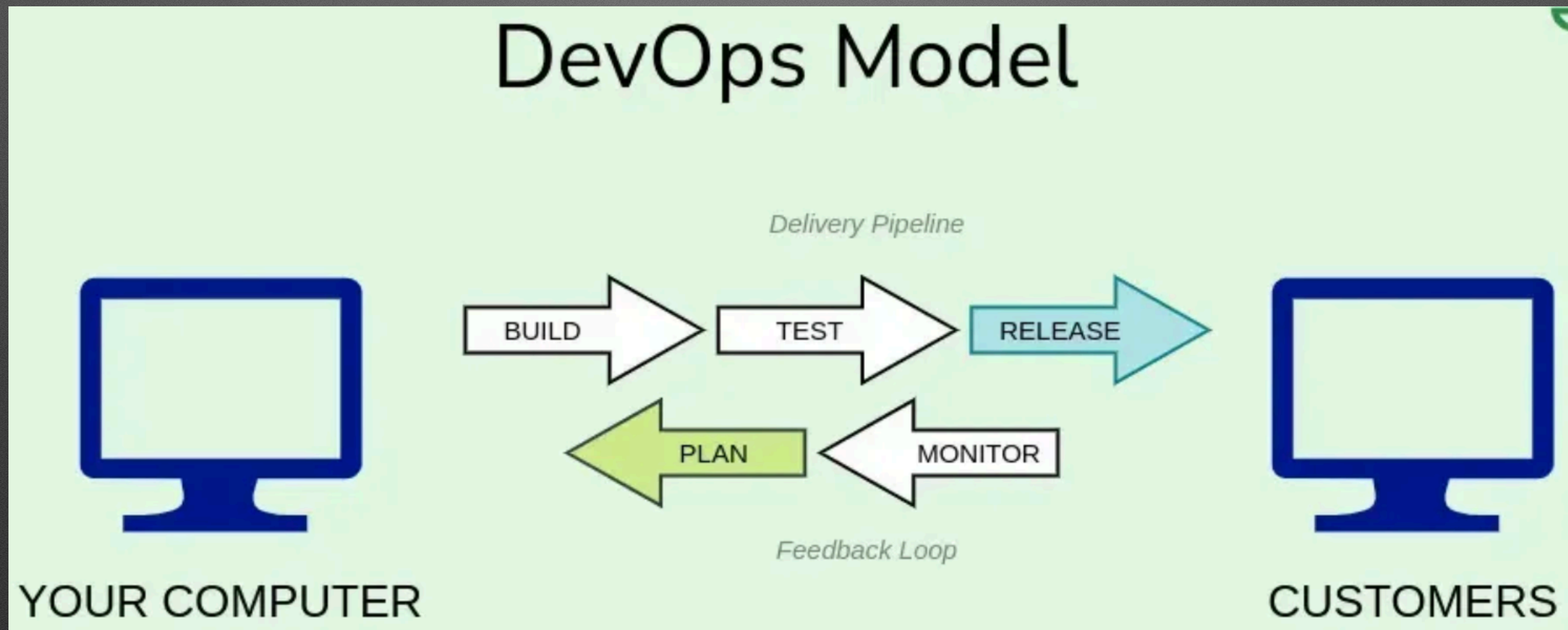


DevOps

Лекція 1

DevOps Model



Build Stage

- Отримання коду та управління версіями
 - Розробники роблять комміти у систему контролю версій (Git).
 - **CI/CD** пайплайн отримує останню версію коду для побудови.
- Компіляція та підготовка коду
 - Код компілюється (для мов, що компілюються, напр. Java, C#).
 - Підготовлюється готовий до тестування код.
- Unit Testing (модульне тестування)
 - Перевіряються окремі функції або модулі.
 - Виявляються базові помилки на ранньому етапі.
- Integration Testing (інтеграційне тестування)
 - Перевіряється взаємодія між компонентами системи у ізольованому середовищі.
- **Пакування артефактів та залежностей**
 - Усі бібліотеки та залежності включаються для створення deployable артефакту (наприклад JAR, Docker image).
- Статичний аналіз коду та перевірка якості (опційно)
 - Аналіз на стиль, потенційні баги, вразливості.
 - Інструменти: SonarQube, ESLint, Ruff, Jenkins, Gradle, Maven, GitLab CI/CD.

Test Stage

- Програмне забезпечення проходить більш детальне тестування, щоб виявити баги, проблеми продуктивності та уразливості безпеки перед релізом.
- Методи тестування включають:
- Performance Testing (тестування продуктивності): вимірювання швидкості та масштабованості.
- Security Testing (тестування безпеки): виявлення потенційних вразливостей.
- Популярні інструменти: Selenium, JUnit, PyTest, Testcontainers, Playwright, TestNG, ZAP.

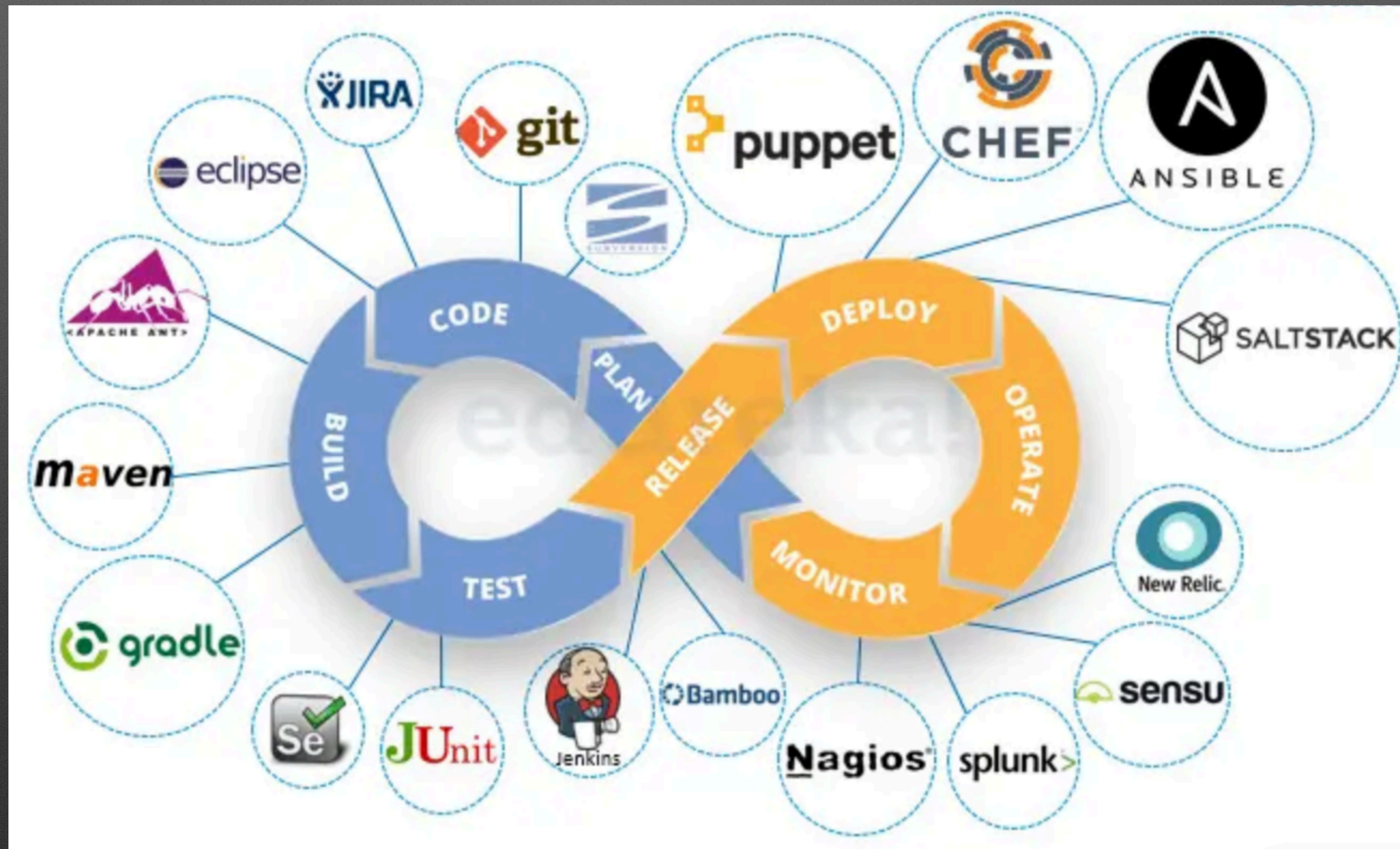
Release Stage

- Програмне забезпечення розгортається у тестовому (staging) середовищі для симуляції реальних умов.
- Якщо все працює коректно, ПЗ розгортається у продуктивне середовище за допомогою стратегій розгортання:
 - Blue-Green Deployment: Два ідентичних середовища чергують трафік для плавного оновлення.
 - Canary Deployment: Нову версію спочатку отримує невеликий відсоток користувачів, щоб перевірити безпечність оновлення.
 - Rolling Updates: Оновлення поступово поширюється на всіх користувачів.
- Популярні інструменти: Docker, Kubernetes, Ansible, Helm.

Continuous Feedback Loop

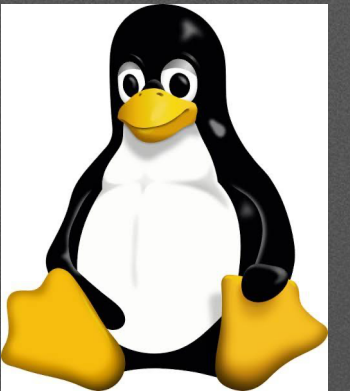
- Ключовий аспект DevOps — навчання на основі реальної роботи системи та використання зворотного зв'язку для покращення майбутніх релізів.
- Моніторинг та логування: Відстеження продуктивності системи та виявлення помилок.
- Зворотний зв'язок від користувачів: Збір відгуків для покращення функціоналу.
- Реагування на інциденти: Системи сповіщень повідомляють команди про збої для швидкого виправлення.
- Оптимізація процесів: Аналіз попередніх релізів для покращення автоматизації та робочого процесу.
- Популярні інструменти: Loki, Prometheus, Grafana, Datadog.

DevOps Lifecycle



<https://medium.com/@imfuzail09/day-1-devops-basics-7cs-of-the-devops-lifecycle-d22d3cab2e99>

Unix/Linux



Unix — це сімейство багатозадачних, багатокористувацьких комп'ютерних операційних систем (ОС), що походять від оригінальної системи, розробленої в Bell Labs у 1960-х і 1970-х роках Кеном Томпсоном і Деннісом Рітчі

Linux - open-source альтернатива Unix, створена Linus Torvalds в 1990-х роках. Відкритий аналог Unix, сумісний із більшістю Unix-програм

Основні характеристики:

🖥️ Багатозадачність — кілька процесів одночасно

👥 Багатокористувацькість — одночасний доступ кількох користувачів

⚙️ Портативність — працює на різних пристроях

🧩 Модульність — ядро + утиліти

Архітектура:

- Ядро (Kernel) — керує ресурсами
- Shell — інтерфейс користувача
- Файлова система — все як файли



Перегляд та обробка тексту

- **cat** [options] [file1 file2 ...] — перегляд вмісту файлу
 - Виводить весь вміст файлу на стандартний вихід (екран).
 - Опції:
 - -n — нумерація рядків
 - -b — нумерація лише непорожніх рядків
 - Приклад: cat -n file.txt
- **less** [file]/ **more** [file] — посторінковий перегляд
 - Дозволяє переглядати великі файли посторінково.
 - less дозволяє переміщення вперед і назад (Space — вперед, b — назад), more тільки вперед.
- **grep** [options] 'pattern' [file1 file2 ...] — пошук по тексту
 - Пошук рядків, що відповідають регулярному виразу або текстовому шаблону.
 - Опції:
 - -i — ігнорувати регістр
 - -r — рекурсивний пошук у каталогах
 - -n — показати номер рядка
 - Приклад: grep -i 'error' /var/log/syslog
- **awk** 'actions' [file] — обробка текстових файлів
 - Дії задаються у фігурних дужках {}: вибір колонок, обчислення, форматування.
 - \$1, \$2, ... — посилання на колонки.
 - Приклад: awk '{print \$1, \$3}' file.txt — виведе 1-й та 3-й стовпець.
awk -F',' 'NR>1 {print \$2}' file.csv - пропустити заголовок у csv-файлі
awk '/2024-01-01/,/2024-01-31/ {print}' log.txt - обробити блок між датами
- **sed** [options] 's/pattern/change/g' [file] — пошук та заміна у файлах
 - Опції:
 - -i — редагування файлу на місці
 - -e — виконати вираз
 - Приклад: sed -i 's/old/new/g' file.txt — замінює всі входження old на new у файлі.
- **cut** [options] -d 'delimiter' -ffield_number [file] — вибір колонок з файлів
 - Опції:
 - -d — символ роздільника колонок
 - -f — номер/номери колонок
 - Приклад: cut -d',' -f1,3 file.csv — виведе перший і третій стовпець CSV.

Системна інформація

- **top [-u user]** — моніторинг ресурсів. Показує CPU, RAM, процеси у реальному часі.
- **df [options] [file_system]** - показує зайняте та вільне місце на файлових системах.

- Опції:

- -h — показати у зручному вигляді (GB, MB)

- Приклад: df -h

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/
tmpfs	2.0G	100M	1.9G	5%	/run

- **du [options] [path]** — розмір файлів/каталогів

- Опції:

- -h — зручний формат

- -s — підсумковий розмір каталогу

- Приклад: du -sh /home/user/Documents

1.2G /home/user/Documents

- **free [options]** — показує зайняту, вільну та буферну пам'ять.

- Опції:

- -h — у зручному для людини форматі (MB, GB)

- Приклад: free -h

	total	used	free	shared	buff/cache	available
Mem:	8.0G	4.0G	2.5G	0.5G	1.5G	3.0G
Swap:	2.0G	0.0G	2.0G			

Архівація та стиснення

tar [options] archive_name.tar files... —
архівація

- **tar -cvf archive.tar file1 file2**
 - -c — створити архів
 - -v — виводити процес створення
 - -f — вказати ім'я архіву
- **tar -xvf archive.tar**
 - -x — розпакувати архів

gzip [options] file / gunzip [options] file.gz —
стиснення/розпаковка

- **gzip file.txt** → створюється file.txt.gz
(оригінальний видаляється, -k залишає оригінал)

- **gzip file1, file2** → створяться file1.gz та file2.gz
- **gunzip file.txt.gz** → повертає file.txt
- **tar -czvf archive.tar.gz myfolder/** →
Створення архіву каталогу myfolder у форматі tar.gz (поєднання tar та gzip)

zip [options] archive.zip files... / unzip archive.zip —
стиснення/розпаковка ZIP файлів

- Приклад: **zip archive.zip file1 file2**
- Розпакування: **unzip archive.zip**

Мережа (1)

ping [options] host — перевіряє, чи відповідає машина на службові пакети ICMP. Не перевіряє, чи працює сервіс на ній. У першому рядку виводу видно IP-адресу, на яку резолвилось ім'я. Тобто це найшвидший спосіб перевірити DNS. -c <num> — кількість пакетів (без цього виконується нескінченно)

- ping -c 2 google.com

curl [options] URL - інструмент для роботи з HTTP-запитами (підтримує GET, POST, PUT, DELETE тощо).

- curl http://example.com — GET-запит.
- curl -X POST -d "name=John&age=25" http://example.com/form — POST із формою.
- curl -X POST -H "Content-Type: application/json" -d '{"user": "Alice"}' http://example.com/api — POST із JSON.

wget [options] URL - завантаження файлів із мережі.

- -O file — збереження під іншим ім'ям.

- wget -O page.html http://example.com

ip [options] object [command] - управління мережевими інтерфейсами, IP-адресами, маршрутами, тунелями та ін.

- Основні об'єкти:
 - link — робота з мережевими інтерфейсами (увімкнення/вимкнення, параметри).
 - addr — управління IP-адресами.
 - route — управління таблицею маршрутів.
- ip addr show # показати IP-адреси всіх інтерфейсів

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500  
inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
```

- ip link set eth0 up # увімкнути інтерфейс eth0
- ip addr add 192.168.1.10/24 dev eth0 # додати IP-адресу
- ip route show # показати таблицю маршрутів

Мережа (2)

nc [options] host port — перевіряє, чи приймає з'єднання конкретний порт на віддаленій машині.

Основні опції

- **-z** — «zero-I/O»: тільки перевірити, чи порт відкритий, не надсилаючи даних
- **-v** — verbose: вивести результат у stderr. Без нього дізнатися результат можна тільки з коду виходу (echo \$?)
- **-w <n>** — таймаут у секундах.

Розрізняє два випадки

connection refused — машина відповіла, але на цьому порту ніхто не слухає. Процес не запущений або слухає інший порт.

тиша до тайм-ауту — відповіді немає взагалі. Пакети мовчки відкидає мережевий екран або правило доступу.

```
nc -zv example.com 8080
```

```
nc: connect to example.com port 8080 (tcp) failed:
```

Connection refused

```
nc -zvw 3 example.com 3306
```

```
nc: connect to example.com port 3306 (tcp) failed:
```

Operation timed out

ss [options] — показує сокети на цій машині: хто що слухає і які з'єднання відкриті. Замінив застарілий netstat.

Основні опції

- **-t** — TCP
- **-l** — тільки ті, що слухають
- **-n** — числа замість імен (швидше й точніше)
- **-p** — який процес

```
ss -tlnp # усі відкриті порти цієї машини
```

```
ss -tlnp | grep 8000 # хто зайняв конкретний порт
```

Навіщо: відповідає на два щоденні питання — «мій сервіс справді слухає?» і «хто зайняв порт, через якого я не можу запуститися».

Логування та перенаправлення

Перенаправлення виводу:

- **>** — перенаправлення стандартного виводу (stdout, дескриптор 1) у файл (перезапис)
echo "Hello" > file.txt
- **>>** — додавання стандартного виводу у файл (без перезапису)
echo "World" >> file.txt

Перенаправлення помилок:

- **2>** — перенаправлення стандартного потоку помилок (stderr, дескриптор 2)
ls nonexistent 2> error.log
- **2>&1** — об'єднання помилок та стандартного виводу
rm some.log >>deletion.log 2>&1

Логування у системний журнал:

- **logger** — запис повідомлень у системний журнал (/var/log/syslog)
logger "Some message"

Ротація логів (logrotate)

загальна конфігурація: /etc/logrotate.conf

створити свої правила

файл /etc/logrotate.d/deletion.log:
/path/to/deletion.log {

```
# rotate once a day
daily
# keep 7 old files
rotate 7
# compress rotated files
compress
# keep the most recent one uncompressed — it is still being
read
delaycompress
# do not fail if the file is missing
missing
# skip rotation if the file is empty
notifempty
# name rotated files as deletion.log-20260901
dateext
# copy the content and truncate the original,
# so the writing process keeps its file descriptor
copytruncate
}
```

Unix/Linux Shells

- Інтерфейс між користувачем і ядром ОС
- Виконує команди та скрипти, дозволяє автоматизувати задачі
 - sh (Bourne shell) - класичний Unix shell, основа для багатьох інших
 - csh (C shell) - схожий на мову C, зручний для програмістів
 - Bash (Bourne Again SHell) - найпопулярніший у Linux, сумісний з Bourne shell, потужні скрипти
 - zsh - розширений shell з автодоповненням, темами, популярний у macOS і Linux

Scripts

`#!/bin/bash`

- <https://devhints.io/bash>
- <https://github.com/RehanSaeed/Bash-Cheat-Sheet>

Цикли

for NAME [in WORDS ...] ; do COMMANDS; done

```
for file in /home/user/docs/*; do
```

```
    echo "Found file: $file"
```

```
done
```

```
for process in $(ps -e | awk '{print $1}'); do
```

```
    echo "Process ID: $process"
```

```
done
```

while [CONDITION]; do COMMAND; done

while CONTROL_COMMAND; do COMMANDS; done

```
ls /home/user/docs | while read file; do
```

```
    echo "File: $file"
```

```
done
```

```
while [ ! -d directory_expected ]
```

```
do
```

```
    echo "`date` - Still waiting"    echo "$(date) - Still waiting"
```

```
    sleep 1
```

```
done
```

```
echo "DIRECTORY IS THERE!!!"
```

until [CONDITION]

do

[COMMANDS]

done

```
until [ -f /home/user/docs/ready.txt ]; do
```

```
    echo "Waiting for ready.txt to appear..."
```

```
    sleep 1
```

```
done
```

```
echo "File is ready!"
```

Умовні оператори

```
if COMMANDS; then COMMANDS; [ elif  
COMMANDS; then COMMANDS; ]... [ else  
COMMANDS; ] fi
```

```
FILE="/home/user/docs/report.txt"
```

```
if [ -f "$FILE" ]; then
```

```
    echo "File exists"
```

```
else
```

```
    echo "File doesn't exist"
```

```
fi
```

```
case EXPRESSION in
```

```
    Pattern_Case_1)
```

```
        STATEMENTS
```

```
;;
```

```
    Pattern_Case_N)
```

```
        STATEMENTS
```

```
;;
```

```
*)
```

```
    STATEMENTS ;;
```

```
esac
```

```
read -p "Enter action (list/delete): " action
```

```
case "$action" in
```

```
    list) ls /home/user/docs ;;
```

```
    delete) rm -f /home/user/docs/*.tmp ;;
```

```
    *) echo "Unknown action" >&2; exit 1 ;;
```

```
esac
```

Що використовуємо в умовах

1. Файли та директорії:

- f FILE — файл існує і є звичайним файлом
- d DIR — директорія існує
- e FILE — файл або директорія існує
- r FILE — файл доступний для читання
- w FILE — файл доступний для запису
- x FILE — файл виконуваний

2. Рядки:

- = — рядки рівні
- != — рядки не рівні
- z STRING — рядок порожній
- n STRING — рядок непорожній

3. Числа:

- eq — рівні
- ne — не рівні
- lt — менше
- le — менше або рівне
- gt — більше
- ge — більше або рівне

4. Логічні оператори:

- ! — заперечення
- && — логічне “і”
- || — логічне “або”

Що використовуємо в умовах

```
FILE="some.log"  
NUMBER=5  
STRING="hello"
```

```
if [ -f "$FILE" ]; then  
    echo "Файл існує"  
fi
```

```
if [ -d "/home/user/docs" ]; then  
    echo "Директорія існує"  
fi
```

```
if [ "$STRING" = "hello" ]; then  
    echo "Рядок дорівнює hello"  
fi
```

```
if [ -z "$STRING" ]; then
```

```
    echo "Рядок порожній"  
fi  
if [ "$NUMBER" -gt 3 ]; then  
    echo "Число більше 3"  
fi
```

```
if [ "$NUMBER" -le 10 ]; then  
    echo "Число менше або рівне 10"  
fi
```

```
if [ "$NUMBER" -gt 0 ] && [ "$NUMBER" -lt 10 ]; then  
    echo "Число від 1 до 9"  
fi
```

```
if [[ $STRING = "hello" || $STRING = "hi" ]]; then  
    echo "Рядок hello або hi"  
fi
```

bash

Cron

cron — сервіс для автоматичного запуску команд та скриптів за розкладом

Використовується для регулярних завдань: резервне копіювання, очищення логів, оновлення даних

Таблиця cron (crontab)

- * * * * * command_to_run
- | | | | |
- | | | | | — день тижня (0-7, де 0 або 7 = неділя)
- | | | | | — місяць (1-12)
- | | | | | — день місяця (1-31)
- | | | | | — година (0-23)
- | | | | | — хвилина (0-59)

Приклади використання

- Щоденний запуск скрипта о 2:30 ранку
- 30 2 * * * /home/user/backup.sh

- Щогодинний запуск
- 0 * * * * /home/user/check_logs.sh

- Кожні 5 хвилин
- */5 * * * * /home/user/sync_data.sh

Основні команди

- crontab -e — редагувати власний crontab
- crontab -l — переглянути поточні завдання
- crontab -r — видалити всі завдання користувача

💡 Поради:

- Переконайтеся, що скрипт має право на виконання (chmod +x script.sh)
- Використовуйте повні шляхи до команд і файлів у cron
- Логи cron зазвичай можна перевірити у /var/log/cron або /var/log/syslog

<https://opensource.com/article/17/11/how-use-cron-linux>

Частина 1: Налаштування локального середовища

1. Git

Налаштування глобальних
параметрів

```
git config --global user.name "name"
```

Перевірка Git

```
git config --global user.email  
"mail@example.com"
```

```
git --version
```

```
git config --global core.editor "code  
--wait"
```

Частина 1: Налаштування локального середовища

2. VS Code

Завантажити VS Code.

Встановити розширення через
Extentions:

- GitLens

- Docker
- YAML
- Markdown

Частина 1: Налаштування локального середовища

3. Термінал

Для Windows — встановити WSL2 з Ubuntu.

Налаштувати інтеграцію з VS Code через Remote - WSL.

Node.js, Python, Docker

```
node -v
```

```
npm -v
```

```
python3 --version
```

```
pip3 --version
```

```
docker --version
```

Клієнт встановлений

```
docker info
```

Демон доступний і працює

Частина 2. GitHub профіль

Створити акаунт GitHub, додати фотографію та опис.

Налаштувати SSH ключ:

1. Створити SSH ключ (якщо немає)

```
ls -al ~/.ssh
```

```
ssh-keygen -t ed25519 -C
```

```
"your_email@example.com" # якщо ключів немає
```

2. Перевірка ssh-agent

```
ps aux | grep ssh-agent
```

```
eval "$(ssh-agent -s)" # якщо агент не  
запущений
```

3. Додати ключ до агента

```
ssh-add ~/.ssh/id_ed25519
```

4. Додати ключ на GitHub

```
cat ~/.ssh/id_ed25519.pub # скопіювати
```

GitHub → Settings → SSH and GPG keys → New
SSH key → вставити → Add SSH key

5. Перевірка з'єднання

```
ssh -T git@github.com
```

При запиті: yes

Успішне повідомлення:

Hi username! You've successfully authenticated...

Частина 3: Портфоліо репозиторій

- Створити публічний репозиторій devops-portfolio з базовою структурою папок. Додати користувача romanenko-natali до цього репозиторію
- Написати професійний README.md з особистою інформацією, цілями в DevOps, планом розвитку та контактами

https://media.datacamp.com/legacy/image/upload/v1697797990/Marketing/Blog/Markdown_Cheat_Sheet.pdf

- Додати .gitignore файл для Node.js, Python, Docker та системних файлів

```
curl https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore -o .gitignore_node
```

```
curl https://raw.githubusercontent.com/github/gitignore/main/Python.gitignore -o .gitignore_python
curl https://raw.githubusercontent.com/github/gitignore/main/Docker.gitignore -o .gitignore_docker
```

```
cat .gitignore_node .gitignore_python .gitignore_docker |
sort | uniq > .gitignore
echo -e "\n# System files\n.DS_Store\nThumbs.db"
>> .gitignore
rm .gitignore_node .gitignore_python .gitignore_docker
git add .gitignore
git commit -m "Add combined .gitignore"
git push origin main
```

Частина 4: Професійний Git workflow

Зробити дві захищених гілки: main та develop

main

- обов'язковий pull request перед злиттям
- заборона force-push
- заборона видалення гілки
- лінійна історія
- Settings → Rules → Rulesets → New branch ruleset
- Target branches: main

Що поки не вмикаємо

- Require status checks — список порожній, поки конвеєр жодного разу не запускався
- Require approvals — у персональному репозиторії ви заблокуєте самі себе: автор PR не може його схвалити
- Do not allow bypassing — з тієї ж причини

develop

- створити від main
- обов'язковий pull request
- заборона force-push

Частина 4: Професійний Git workflow

- Створити GitHub Project "DevOps Learning Journey" з дошкою (Backlog, In Progress, Review, Done)
- Створити Issues для кожного модуля курсу та додати їх до Project
- Налаштувати автоматизацію Project: коли PR мерgetься - issue переміщується в Done, коли PR відкривається - в Review, коли Issue закривається - в Done

В Project → Automation

Правила:

When PR is merged → Move linked issue to Done

When PR is opened → Move linked issue to Review

When Issue is closed → Move issue to Done

Частина 4: Професійний Git workflow

- Додати template для Pull Requests з розділами опису, related issue, checklist та тестування

.github/pull_request_template.md

Опис змін

Коротко опишіть, що було змінено та чому.

Пов'язане завдання

Посилання на відповідне завдання або issue (наприклад, Jira, Trello).

Як перевірити

Кроки для тестування змін.

Скріншоти (якщо є)

Візуальні зміни або інтерфейсні оновлення.

Додаткові зауваження

Будь-яка додаткова інформація для рецензента.

Список перевірок

- [] Код протестовано
- [] Пройдено перевірку стилю
- [] Документація оновлена
- [] Всі тести пройшли успішно

Частина 4: Професійний Git workflow

- Створити Telegram канал та додати повідомлення про створення PR в репозиторії до цього каналу

1. Знайдіть @BotFather в Telegram
2. Надішліть /newbot
3. Дайте ім'я боту та username
4. Збережіть API токен
5. Відкрийте груповий чат
6. Натисніть на назву групи → "Manage Group" → "Administrators"
Натисніть "Add Admin"
7. Знайдіть вашого бота за @username

8. Додайте бота як адміністратора (дайте права "Post Messages")

9. Надішліть повідомлення в групу (будь-яке)

10. Перейдіть за посиланням: https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getUpdates

11. Знайдіть chat.id - для групи це буде від'ємне число (наприклад: -123456789)

Важливо:

Приватний чат: Chat ID позитивне (123456789)

Група: Chat ID від'ємне (-123456789)

Канал: Chat ID починається з -100 (-1001234567890) - саме для каналу потрібні кроки 6-8

Частина 4: Професійний Git workflow

12. Створіть файл .github/workflows/telegram-notify.yml:

yml

name: Telegram Notification

on:

push:

branches: [main]

just for opened PR from any branch

pull_request:

types: [opened]

permissions:

contents: read

jobs:

notify:

runs-on: ubuntu-latest

steps:

- name: Send Telegram notification

uses: appleboy/telegram-action@v1

with:

to: \${{ secrets.TELEGRAM_CHAT_ID }}

token: \${{ secrets.TELEGRAM_BOT_TOKEN }}

message: |

🚀 New commit in repository: \${{ github.repository }}

Commit: \${{ github.event.head_commit.message }}

Author: \${{ github.event.head_commit.author.name }}

View changes: \${{ github.event.head_commit.url }}

just for opened PR from any branch

#Repository: \${{ github.repository }}

#PR Title: \${{ github.event.pull_request.title }}

#Author: \${{ github.event.pull_request.user.login }}

#From: \${{ github.event.pull_request.head.ref }}

#To: \${{ github.event.pull_request.base.ref }}

#View PR: \${{ github.event.pull_request.html_url }}

Частина 4: Професійний Git workflow

- Створити Discord канал та додати повідомлення про створення PR в репозиторії до цього каналу

Через Github WebHook

1. Виберіть канал для сповіщень у Discord сервері
 2. Клікніть на шестерню біля назви каналу → Integrations → Webhooks → Create Webhook
 3. Name: GitHub Notifications (або будь-яке ім'я)
 4. Скопіюйте Webhook URL (виглядає як: `https://discord.com/api/webhooks/123456789012345678/abcdefghijklmnopqrstuvwxyz`)
 5. Відкрийте ваш репозиторій на GitHub та виберіть Settings → Webhooks → Add webhook
- Заповніть форму:

- Payload URL: вставте Discord webhook URL + додайте **/github** в кінці
`https://discord.com/api/webhooks/<id>/<token>/github`
 - Content type: application/json
 - Secret: залиште порожнім (Discord його не перевіряє; у робочих інтеграціях — заповнюють)
 - SSL verification: Enable SSL verification 
6. Виберіть потрібні Events
Let me select individual events → Pull requests
Або просто:
- ☒ Send me everything (отримувати все)
7. Активуйте Webhook
- ☒ Active (переконайтесь що позначено)

Частина 4: Професійний Git workflow

- Створити Discord канал та додати повідомлення про створення PR в репозиторії до цього каналу

Через Github Actions

Крок 1. Створити webhook у Discord
Відкрийте Discord → перейдіть на потрібний сервер/канал
Натисніть шестерню біля назви каналу → "Integrations"
Натисніть "Create Webhook" або "Webhooks" → "New Webhook"

Дайте ім'я webhook (наприклад, "GitHub Notifications")

Скопіюйте Webhook URL (виглядає як: <https://discord.com/api/webhooks/123456789/abcdef...>)

Крок 2. Додати URL у GitHub Secrets

У вашому репозиторії:

Settings → Secrets and variables → Actions

Додайте:

DISCORD_WEBHOOK_URL (URL webhook з Discord)

Частина 4: Професійний Git workflow

- Створити Discord канал та додати повідомлення про створення PR в репозиторії до цього каналу

Через Github Actions

Крок 3: Написати workflow

Створіть файл `.github/workflows/discord-notify.yml`:

```
name: Discord Notification Advanced
```

```
on:
```

```
  pull_request:
```

```
    types: [opened]
```

```
jobs:
```

```
  notify:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
- name: Send Discord Embed
```

```
  uses: sarisia/actions-status-discord@v1
```

```
  with:
```

```
    webhook: ${{ secrets.DISCORD_WEBHOOK_URL }}
```

```
    title: "New Pull Request"
```

```
    description: |
```

```
      **${{ github.event.pull_request.title }}**
```

```
    From: `${{ github.event.pull_request.head.ref }}
```

```
    To: `${{ github.event.pull_request.base.ref }}
```

```
    Author: ${{ github.event.pull_request.user.login }}
```

```
    color: 0x00ff00
```

```
    url: ${{ github.event.pull_request.html_url }}
```

```
    username: "GitHub Bot"
```

```
    avatar_url: "https://github.githubassets.com/images/modules/logos_page/GitHub-Mark.png"
```